Lunar Descent Plant and Flight Software: Mathematics

1 Conventions

SI units throughout: meters, seconds, kilograms, radians. Frame: origin at the Moon's center, z along the Moon's spin axis, axes fixed relative to the stars (Moon-centered inertial). The Moon does not move. Source: `workloads/moon/common/`.

2 Vectors

For $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$ and scalar s:

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z \tag{1}$$

$$\mathbf{a} \times \mathbf{b} = (a_y b_z - a_z b_y,\ a_z b_x - a_x b_z,\ a_x b_y - a_y b_x) \tag{2}$$

$$\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}} \tag{3}$$

The cross product follows the right-hand rule: $\hat{x} \times \hat{y} = \hat{z}$. Code: `vec3.hpp`. Tests: `test_vec3.cpp`.

3 Constants

Symbol	Value	Unit	Source
μ_M	$4.902800066 \times 10^{12}$	$\mathrm{m}^3/\mathrm{s}^2$	JPL DE430
R_M	1 737 400	m	mean radius
ω_M	2.6616995×10^{-6}	rad/s	sidereal spin, one turn per 27.321661 days
g_0	9.80665	m/s^2	standard gravity

The spin vector is $\boldsymbol{\omega}_M = (0, 0, \omega_M)$. The ground under a point $\mathbf{r}$ moves at $\boldsymbol{\omega}_M \times \mathbf{r}$, which is $\omega_M R_M = 4.624$ m/s at the equator. Code: `constants.hpp`.

4 Gravity

With $r = \|\mathbf{r}\|$:

$$\mathbf{g} = -\mu_M \frac{\mathbf{r}}{r^3} \tag{4}$$

At the surface, $\mu_M / R_M^2 = 1.6242$ m/s^2. Earth's tidal acceleration near the Moon is on the order of 10^{-5} m/s^2 (estimated) and is not modeled. Code: `dynamics.cpp`. Tests: `test_dynamics.cpp`.

5 Engine and actuator limits

The flight software commands a throttle τ and a direction $\mathbf{d}$. The plant applies the limits of a real engine:

$$\tau' = \begin{cases} 1 & \tau > 1 \\ \tau & 0 < \tau \leq 1 \\ 0 & \text{otherwise (including NaN)} \end{cases} \tag{5}$$

The engine is off ($\tau' = 0$) when $\|\mathbf{d}\|$ is zero, infinite, or NaN, or when the mass has reached the dry mass m_{dry}. Otherwise:

$$F = \tau' F_{\max} \tag{6}$$

$$\mathbf{a}_T = \frac{F}{m} \frac{\mathbf{d}}{\|\mathbf{d}\|} \tag{7}$$

$$\dot{m} = -\frac{F}{I_{sp}\, g_0} \tag{8}$$

The direction is normalized, so a corrupted command can point the engine anywhere but cannot push harder than full thrust. These limits are physics of the plant, not a flight software defense.

6 Equations of motion

$$\dot{\mathbf{r}} = \mathbf{v} \tag{9}$$

$$\dot{\mathbf{v}} = \mathbf{g}(\mathbf{r}) + \mathbf{a}_T \tag{10}$$

$$\dot{m} = -\frac{F}{I_{sp}\, g_0} \tag{11}$$

The system does not depend on time explicitly. Code: `dynamics.cpp`, function `derivative`.

7 Numerical integration (RK4)

Write the state as $\mathbf{y} = (\mathbf{r}, \mathbf{v}, m)$ and the equations of motion as $\dot{\mathbf{y}} = f(\mathbf{y})$. The classical fourth-order Runge–Kutta step of size h:

$$\mathbf{k}_1 = f(\mathbf{y}_n) \tag{12}$$

$$\mathbf{k}_2 = f(\mathbf{y}_n + \tfrac{h}{2}\mathbf{k}_1) \tag{13}$$

$$\mathbf{k}_3 = f(\mathbf{y}_n + \tfrac{h}{2}\mathbf{k}_2) \tag{14}$$

$$\mathbf{k}_4 = f(\mathbf{y}_n + h\,\mathbf{k}_3) \tag{15}$$

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \tfrac{h}{6}\,(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \tag{16}$$

The engine command is held constant for the whole step, as flight software sets one command per tick. If $m_{n+1} < m_{\text{dry}}$, it is set to m_{dry}; the step still applied thrust for its full length, so the error is at most one step of thrust.

Validation. With no thrust, specific orbital energy $\mathcal{E} = \frac{1}{2}\|\mathbf{v}\|^2 - \mu_M/r$ and angular momentum $\mathbf{h} = \mathbf{r} \times \mathbf{v}$ are conserved. Over one 100 km circular lunar orbit (period 7067.5 s) in 5000 steps, measured drift is 2.1×10^{-15} in $\mathcal{E}$ and 8.9×10^{-16} in $\|\mathbf{h}\|$, at the level of double rounding, and the spacecraft returns to within a millimeter of its start. Forward Euler on the same orbit drifts 1.5×10^{-2} in $\mathcal{E}$ and returns 138 km off. Code: `integrator.cpp`. Tests: `test_integrator.cpp`.

8 Navigation (flight software)

The flight software receives the Moon-centered state $(\mathbf{r}, \mathbf{v})$ and derives the ground-relative state:

$$\mathbf{u} = \mathbf{v} - \boldsymbol{\omega}_M \times \mathbf{r} \qquad \text{velocity relative to the ground} \tag{17}$$

$$\hat{\mathbf{n}} = \mathbf{r}/\|\mathbf{r}\| \qquad \text{local up} \tag{18}$$

$$h = \|\mathbf{r}\| - R_M \qquad \text{altitude} \tag{19}$$

$$v_\uparrow = \mathbf{u} \cdot \hat{\mathbf{n}} \qquad \text{vertical speed, positive up} \tag{20}$$

$$v_\leftrightarrow = \|\mathbf{u} - v_\uparrow \hat{\mathbf{n}}\| \qquad \text{horizontal speed} \tag{21}$$

This stage runs before guidance, so a fault here reaches every TMR replica.

Limitations. The real lunar spin axis is tilted about 1.5° from the orbit normal (Cassini's laws); the model fixes it along z. Altitude is above a sphere of radius R_M; there is no terrain. Code: `fsw/navigation.cpp`. Tests: `test_navigation.cpp`.

9 Guidance (flight software)

Inputs: the navigation state and the mass m. Output: throttle τ and direction $\mathbf{d}$; the plant applies the actuator limits.

Braking. Full throttle, $\tau = 1$, split between altitude control and horizontal deceleration. With $a_{\max} = F_{\max}/m$, $w = v_\leftrightarrow$, $r = R_M + h$, and $h_b = 1500$ m:

$$t_{\text{stop}} = w/a_{\max} \tag{22}$$

$$v_b^* = -\frac{h - h_b}{\max(t_{\text{stop}},\ 1\text{ s})} \tag{23}$$

$$a_\uparrow = \text{clamp}\left(\frac{\mu_M}{r^2} - \frac{w^2}{r} + k_v\,(v_b^* - v_\uparrow),\ 0,\ a_{\max}\right) \tag{24}$$

$$a_\leftrightarrow = \sqrt{(a_{\max} - a_\uparrow)(a_{\max} + a_\uparrow)} \tag{25}$$

$$\mathbf{d} = a_\uparrow \hat{\mathbf{n}} - \frac{a_\leftrightarrow}{w}\,(\mathbf{u} - v_\uparrow \hat{\mathbf{n}}) \tag{26}$$

The sink-rate target brings the lander to h_b as its horizontal speed reaches zero. w^2/r is the support still provided by orbital motion. The factored form of $a_\leftrightarrow$ cannot go negative by rounding, because the clamp guarantees $a_\uparrow \le a_{\max}$. Pure retrograde braking from 15 km (no altitude control) was measured to strike the surface at 132 m/s with 726 m/s of horizontal speed remaining.

Switch. Braking ends, permanently, when $v_\leftrightarrow < 50$ m/s or $h < 500$ m, whichever comes first.

Final descent. Target vertical speed and wanted acceleration:

$$v^* = -\operatorname{clamp}\left(\frac{h}{10\ \mathrm{s}},\ 1,\ 30\right)\ \mathrm{m/s} \tag{27}$$

$$g = \frac{\mu_M}{(R_M + h)^2} \tag{28}$$

$$\mathbf{a}^* = (g + k_v\,(v^* - v_\uparrow))\,\hat{\mathbf{n}} - k_h\,(\mathbf{u} - v_\uparrow \hat{\mathbf{n}}) \tag{29}$$

with $k_v = 1.0\ \mathrm{s}^{-1}$ and $k_h = 0.5\ \mathrm{s}^{-1}$. If $\mathbf{a}^* \cdot \hat{\mathbf{n}} \le 0$ the engine is off (a lander cannot thrust toward the ground). Otherwise

$$\tau = \frac{\|\mathbf{a}^*\|\, m}{F_{\max}}, \qquad \mathbf{d} = \mathbf{a}^* \tag{30}$$

and the plant saturates τ at 1. On the target descent rate with no drift, $\tau = g\,m/F_{\max}$, the hover throttle.

Clean landing. Measured: touchdown at 527.8 s at 1.000 m/s descent and 0.000 m/s horizontal speed, with 1157.6 kg of propellant left; score 0.328. Two clean runs produce byte-identical traces. Code: `fsw/guidance.cpp`. Tests: `test_guidance.cpp`.

10 Simulation and landing judge

Start. Circular equatorial prograde orbit at $h_0 = 15\,\mathrm{km}$: $\mathbf{r}_0 = (R_M + h_0, 0, 0)$, $\mathbf{v}_0 = (0, \sqrt{\mu_M/(R_M + h_0)}, 0)$, $m_0 = 15\,200$ kg. The flight software and the plant step together at $\Delta t = 0.1$ s, for at most 3600 s.

Independence. The judge computes altitude and ground-relative velocity with the same formulas as navigation, in uninstrumented plant code. If it reused the flight software's navigation, a fault there would corrupt the judge as well.

Touchdown. When altitude changes sign within a step, from $h_a > 0$ to $h_b \le 0$, the crossing fraction is $f = h_a/(h_a - h_b)$, and touchdown speeds, time, and position are interpolated linearly at f.

Ground-fixed spot. With $\alpha = -\omega_M t$, the touchdown point is rotated back to the ground's orientation at $t = 0$:

$$\mathbf{s} = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix} \mathbf{r} \tag{31}$$

so two runs landing on the same ground spot at different times give the same $\mathbf{s}$.

Score. With descent speed v, horizontal speed w, and distance d from the clean run's spot (zero for the clean run itself):

$$v_{\lim}(w) = 3.05 - (3.05 - 2.13)\frac{w}{1.22}, \qquad S = \max\left(\frac{v}{v_{\lim}(w)},\ \frac{w}{1.22},\ \frac{d}{1000}\right) \tag{32}$$

in m/s and m (ADR-0020). $S \le 1$ passes. No touchdown within the time limit gives $S = \infty$. The plant reports S as the run's final result. Code: `plant/judge.cpp`, `sim/main.cpp`. Tests: `test_judge.cpp`.

11 Defense: TMR and lockstep hybrid

Modeled on real rocket flight computers: three computers, each a lockstep pair of cores running the full flight software (navigation and guidance). Computer i runs cores $2i$ and $2i+1$, $i \in \{0,1,2\}$, producing commands c_{2i} and c_{2i+1}.

Lockstep inside a computer. Computer i is healthy when its two commands are bit-identical: $h_i = [\, c_{2i} \equiv c_{2i+1} \,]$, comparing the 64-bit patterns of the throttle and the three direction components.

TMR across computers. Let $H = \{i : h_i\}$. Computer $i \in H$ wins if more than half of H produced the same command:

$$2 \left| \{\, j \in H : c_{2j} \equiv c_{2i} \,\} \right| > |H| \tag{33}$$

If no computer wins (two healthy computers that disagree), the one with the longer record of consecutive trusted ticks is used. If H is empty, the longest record is used and the tick is flagged DETECTED. A computer is trusted when it is healthy and agrees with the chosen command; any untrusted computer flags the tick CORRECTED.

Resynchronization. After each tick every core's phase memory (braking or final) is set to the chosen computer's.

Separate cores. The build compiles the per-core sources once per core and internalizes each copy, so every core has its own operations: 6 copies of 49 operations. The comparisons use integer bit patterns, outside the floating-point fault model, so the voter is assumed reliable. Code: `fsw/redundancy.cpp`, `fsw/fsw.cpp`, `fsw/core.cpp`. Tests: `test_redundancy.cpp`.